Guía técnica: Buenas prácticas para el control de versiones

Elaborado por, Jorge González R.

1. Convención de commits

Un mensaje de commit sin convención (p.e. "varios cambios", "algunos arreglos") no aporta detalle acerca del cambió incorporado al desarrollo ni por qué se realizó. Una convención permite:

- Entender el historial sin abrir cada diff.
- Generar “change logs” automáticamente.
- Facilitar la revisión de código.
- Que todos los miembros del equipo de desarrollo documenten los cambios al código fuente usando mensajes homogéneos.

Reglas rápidas:

- Estructura sugerida para un *commit* correctamente documentado:

  <tipo>(<alcance opcional>): <descripción corta en presente>

- Escribir una descripción corta de máximo 50 caracteres aproximadamente, en minúscula y en modo imperativo ("agregar", no "agregado" ni "agrega").
- Hay que recordar que un commit es igual a un cambio lógico (atómico).
- Los principales tipos que se usan en las convenciones son:

  feat: nueva funcionalidad
  fix: corrección de error
  docs: cambio de documentación
  test: creación o ajuste de pruebas
  refactor: mejora interna sin cambiar comportamiento esperado
  style: formato o estilo sin impacto funcional
  build: cambios de dependencias, empaquetado o compilación
  ci: cambios en automatización o integración continua
  chore: tareas de mantenimiento
  perf: mejora de rendimiento

- El alcance es opcional e indica el módulo afectado, p.e.:

  feat(auth): agregar login con JWT
  fix(inventario): corregir cálculo de stock negativo
  docs(readme): agregar instrucciones de instalación

2. Estrategia de ramas

Definan y apliquen una estrategia, por ejemplo, en la cual diferencien la rama de *features* (asociada a las historias de usuario) de la rama de *bugs*/*fixes* y otras que considere el equipo técnico que desarrolla el proyecto:

```
main
 |
 +-- feature/hu01
 |
 +-- feature/hu02
 |
 +-- feature/hu<numero>
```

Cada miembro del equipo deberá participar en el desarrollo con las historias de usuario que le han sido asignadas dentro del sprint backlog mediante una rama de trabajo.

La rama deberá relacionarse con una historia de usuario o una tarea concreta (de una hu) dentro del sprint en curso.

Ejemplo:

HU-15: Registrar producto → feature/hu15-registrar-producto

Recuerden que esa relación entre historia de usuario (HU), rama y commits deberá ser trazable.

3. Pull Request y revisión de código

Cada funcionalidad desarrollada deberá integrarse con la rama principal main/master mediante un *Pull Request (PR)*.

El flujo esperado para ejecutar un PR, puede verse así:

```
feature/hu<id><funcionalidad>
      |
   Pull Request
      |
     main
      |
   Code Review
      |
   Aprobación del revisor
      |
     merge
```

La evidencia de un PR debe demostrar:

- Que el PR existió y se fusionó (no quedó abierto ni se cerró sin mezclar).
- Que hubo revisión real de otra persona (comentarios, cambios solicitados, capturas de pantalla con alguna evidencia de funcionalidad implementada, aprobación).
- Que el proceso quedó registrado en el historial de Git, no solo en la interfaz de GitHub.

Tipos de evidencia y cómo generarlas:

A. Captura de pantalla de la conversación del PR.

   ¿Qué mostrar?

Título, descripción, autor, estado Merged (morado), comentarios del revisor, botón "Approve" o "Changes requested".

¿Cómo obtenerla?

- Entrar al PR en GitHub: https://github.com/<usuario>/<repo>/pull/<n>
- Pestaña *Conversation*: capturar el encabezado con el estado Merged, la descripción y al menos un comentario de revisión visible.
- Pestaña *Files changed*: capturar un comentario dejado sobre las distintas líneas específicas del código a las que haya lugar referirse (evidencia de revisión granular).

B. Captura del historial de aprobaciones (reviewers).

En el panel derecho del PR en Github aparece *Reviewers* con el ícono verde de "Approved". Es conveniente capturar esa sección junto con la fecha, para demostrar que quien aprobó no es el mismo autor del PR.

C. Historial de commits del PR con.

Muestra que el PR abarcó varios commits y cómo se integró a la rama principal main/master.

Comando de referencia:

```
git log --oneline --graph --all
```

También puede usar otros comandos de GitHub CLI o GitHub API, lo que mejor considere.

Si desea mostrar qué cambió exactamente dentro del PR:

```
git diff main...feature/carrito-ventas
```

Si desea mostrar los aportes por integrante, por ejemplo, para demostrar que la revisión fue cruzada:

```
git shortlog -sn --all
```

Para contar commits por persona, y así respaldar que cada integrante tuvo PR propios revisados por otro compañero (no auto-aprobados):

```
git log --author="nombre" --oneline | wc -l
```

D. Captura de *Branch Protection* activa.

En el repo GitHub, ir a las opciones:

Settings → Branches → Branch protection rules → main/master

Capturar la activación de regla con "*Require a pull request before merging*" y "*Require approvals*" marcados. Esta es la evidencia de que la revisión no fue opcional sino forzada por configuración.

E. Captura del tablero de PR.

En el repo GitHub, en la pestaña Pull requests, filtrar por:

is:pr
is:merged

... y capturar la lista completa, mostrando el número total de PR fusionados.

4. Tabla resumen con PR's

Para obtener el enlace de un PR desde la interfaz de GitHub, primero, abrir el PR en GitHub y copiar la URL de la barra del navegador. No se debe enlazar la rama, el commit ni el repositorio general, sino específicamente la URL de la página del PR específico. A continuación, un ejemplo de referencia:

#PR	HU relacionada	Autor	Revisor	Método de merge	Enlace
5	HU-03 Registrar producto (issue #3)	Ana	Luis	Squash and merge	https://github.com/equipo-x/sisteminv/pull/5
6	HU-07 Alerta de stock bajo (issue #7)	Luis	Carla	Merge commit	https://github.com/equipo-x/sisteminv/pull/6
7	HU-05 Carrito de ventas (issue #5)	Carla	Ana	Rebase and merge	https://github.com/equipo-x/sisteminv/pull/7

5. Identificar y resolver conflictos al fusionar ramas

Con el propósito que experimenten el impacto de encontrar conflictos al momento de fusionar dos ramas, conviene dentro del ejercicio académico simular un conflicto como una práctica necesaria dentro de un entorno controlado. Aprender a resolver ese tipo de conflictos es parte del flujo en la puesta en práctica de un esquema de cambios gobernados mediante Pull Request (PR). Un PR con conflictos no se puede fusionar hasta resolverlos, nos enseña método y orden en un Proyecto Informático.

¿Cómo se documenta la resolución de un conflicto de fusión?

- Describan qué archivo y qué líneas entraron en conflicto
- Indiquen los comandos que usaron para identificarlo
- Mencionen cómo decidieron qué cambios conservar y cómo verificaron que todo funcionaba después.
- Si no lograron resolverlo dentro del sprint, expliquen qué falló y qué alternativas están pendientes por probar.

¿Cómo se puede simular un conflicto de fusión entre ramas? (pueden ensayar otras formas, también)

- Dos (2) integrantes crean sus ramas *feature* en una misma línea base (misma versión) de main/master.
- Ambos modifican la misma línea de un archivo.
- El primero abre su PR y se fusiona.
- El segundo actualiza su rama (*git fetch origin* y luego *git merge origin/main* o *git rebase origin/main*), identifica el conflicto y lo resuelve, conservando lo correcto de ambos cambios.
- Ejecuta *git status* para revisar, agrega solo los archivos resueltos (no git add .), hace el *commit* y sube la rama.
- Su PR queda sin conflictos y se fusiona.